

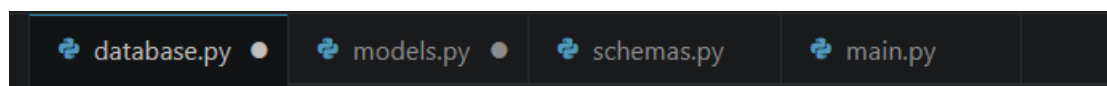
Hospital Staff Management System

Task 1: Create PostgreSQL database.

▼ hospital_db

The screenshot shows the Postman interface with a GET request to `http://127.0.0.1:8000/` executed successfully. The response is a 200 OK status with a response time of 57 ms and a body size of 170 B. The response body is a JSON object with a `message` property set to `"Database Connected Successfully"`.

- database.py
- models.py
- schemas.py
- main.py



Task 3: Create SQLAlchemy model:

database.py models.py X schemas.py main.py

models.py > HospitalStaff

```
1 from sqlalchemy import Column, Integer, String
2 # It is just a table design
3
4 from database import Base
5
6 class HospitalStaff(Base):
7
8     # class used to define structure of table
9
10     __tablename__ = "HospitalStaff"
11
12     id = Column(Integer, primary_key= True)
13
14     username = Column(String)
15
16     password = Column(String)
17
18     role = Column(String)
19
20     department = Column(String)
21
22
```

id [PK] integer	username character varying	password character varying	role character varying	department character varying		

POST

http://127.0.0.1:8000/hospitalstaff/5/Karan/karan@123/Front Desk/Reception

Docs

Params

Authorization

Headers 7

Body

Scripts

Tests

Settings

Query Params

Key	Value
Key	Value

Body

Cookies

Headers 4

Test Results

{ } JSON

Preview

Visualize

1 {

2 "message": "Hospitalstaff Added Successfully"

3 }

Data Output

Messages

Notifications

Showing rows: 1 to 5

	id [PK] integer	username character varying	password character varying	role character varying	department character varying
1	1	Nitesh	nitesh@123	Director	Clinical
2	2	Shiksha	shiksha@123	Doctor	Support & Diagnos...
3	3	Vivek	vivek@123	Doctor	Cardiology
4	4	Rupesh	rupesh@123	Doctor	Padiatrics
5	5	Karan	karan@123	Front Desk	Reception

Task 4: Create Pydantic Models for:

- User Registration
- User Login

- User Registration
- User Login

Schemas:

```
from pydantic import BaseModel
# BaseModel, is used to define the structure of data and validate input data

class LoginSchema(BaseModel):

    username : str

    password : str
```

Main:

```
@app.post("/register_staff")

def add_hospitalstaff(stf: HospitalStaffSchema):
    # staff must follow the structure defined in HospitalStaffSchema
    # i.e. it accepts JSON data and validate it using Pydantic model

    hospitalstaff = HospitalStaff(id = stf.id, username = stf.username, password = stf.password, role = stf.role, department = stf.department)
    # access Hospitalstaff table, find hospitalstaff whose ID matches, and return first matching row

    session.add(hospitalstaff)

    session.commit()

    return{"message": " Hospitalstaff Registered Successfully Using Pydentic Model"}

# get api
❖
@app.get("/staff_login")

def get_hospitalstaff():

    hospitalstaff = session.query(HospitalStaff).all()

    return hospitalstaff
```

Task 5: Create User Registration API Requirements:

- Register new staff members
- Store username, password, role and department

POST http://127.0.0.1:8000/register_staff

Docs Params Authorization Headers 8 Body Scripts Tests Settings

☐ none
 ☐ form-data
 ☐ x-www-form-urlencoded
 ☒ raw
 ☐ binary
 ☐ GraphQL
 JSON

```

1 {
2   "id" : 6,
3   "username" : "Ram",
4   "password" : "ram@123",
5   "role" : "Coordinator",
6   "department" : "Reception"
7 }
```

Body Cookies Headers 4 Test Results

200 OK • 16 ms

{ } JSON Preview Visualize

```

1 {
2   "message": " Hospitalstaff Registered Successfully Using Pydentic Model"
3 }
```

	id [PK] integer	username character varying	password character varying	role character varying	department character varying
1	1	Nitesh	nitesh@123	Director	Clinical
2	2	Shiksha	shiksha@123	Doctor	Support & Diagnos...
3	3	Vivek	vivek@123	Doctor	Cardiology
4	4	Rupesh	rupesh@123	Doctor	Padiatrics
5	5	Karan	karan@123	Front Desk	Reception
6	6	Ram	ram@123	Coordinator	Reception
7	7	Rohan	rohan@123	Doctor	Orthopedics

Task 6: Implement Password Hashing Requirements:

- Hash password before storing it in the database
- Use bcrypt hashing

POST



http://127.0.0.1:8000/register_hashed

Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```
1 {  
2   "id" : 6,  
3   "username" : "Ram",  
4   "password" : "ram@123",  
5   "role" : "Coordinator",  
6   "department" : "Reception"  
7 }
```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize **JSON**

```
1 {  
2   "detail": "Username already exists"  
3 }
```

POST ⌵ http://127.0.0.1:8000/register_hashed

Params Authorization Headers 8 **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ⌵

```

1  {
2    "id" : 8,
3    "username" : "Sonam",
4    "password" : "sonam@123",
5    "role" : "Doctor",
6    "department" : "Neurology"
7  }

```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize **JSON** ⌵ ≡

```

1  {
2    "message": "Staff Registered Successfully"
3  }

```

Showing rows: 1 to 8 ✎ 📄 Page No: 1 of 1						
	id [PK] integer ✎	username character varying ✎	password character varying ✎	role character varying ✎	department character varying ✎	
1	1	Nitesh	nitesh@123	Director	Clinical	
2	2	Shiksha	shiksha@123	Doctor	Support & Diagnos...	
3	3	Vivek	vivek@123	Doctor	Cardiology	
4	4	Rupesh	rupesh@123	Doctor	Padiatrics	
5	5	Karan	karan@123	Front Desk	Reception	
6	6	Ram	ram@123	Coordinator	Reception	
7	7	Rohan	rohan@123	Doctor	Orthopedics	
8	8	Sonam	\$2b\$12\$ocKMr3Aak7cTH2h/00zyNeEeS6welyK7wGpM.xnqQgbaaac2kw...	Doctor	Neurology	

Task 7: Create Login API Requirements:

- Verify username
- Verify password

POST http://127.0.0.1:8000/login

Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "username" : "Ram",
3   "password" : "ham@123"
4 }
5 }
```

Body Cookies Headers 4 Test Results 401 Unauth

Pretty Raw Preview Visualize JSON

```
1 {
2   "detail": "Incorrect Password"
3 }
```

POST http://127.0.0.1:8000/login

Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "username" : "Ram",
3   "password" : "ram@123"
4 }
5 }
```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Login Successful"
3 }
```


- Generate JWT token after successful login

```
token = create_token({"username":user.username})

# create a JWT token and store the username inside the token payload
# the username is stored in the token so that user does not need to login for every request

return {"access_token": token}

# return the generated token to the user after successful login
```

Task 8: Implement JWT Authentication Requirements:

- Create JWT token generation function
- Create JWT verification function

```
from jose import jwt

SECRET_KEY = "mysecretkey"
# secret key is used to create and verify JWT tokens
# "mysecretkey" is the secret password used to sign and verify JWT tokens.

ALGORITHM = "HS256"
# a method used to encrypt/ sign the token
```

Password is incorrect so showing incorrect password:

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8000/login_jwt`. The request body is a JSON object: `{ "username": "Sonam", "password": "onam@123" }`. The response body is a JSON object: `{ "detail": "Incorrect Password" }`. The interface includes tabs for Params, Authorization, Headers, Body, Scripts, and Settings. The Body tab is selected, and the response is displayed in a Pretty view.

```
POST http://127.0.0.1:8000/login_jwt

{
  "username": "Sonam",
  "password": "onam@123"
}
```

```
{
  "detail": "Incorrect Password"
}
```

Hashed password cannot fetched by the normal password so showing incorrect password:

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8000/login_jwt`. The request body is a JSON object with `"username": "Sonam"` and `"password": "sonam@123"`. The response body is a JSON object with `"detail": "Incorrect Password"`. The status bar at the bottom indicates a 200 OK response.

```
POST http://127.0.0.1:8000/login_jwt

{
  "username": "Sonam",
  "password": "sonam@123"
}
```

```
{
  "detail": "Incorrect Password"
}
```

Hashed password fetched by the hashed password so it showing access token:

The screenshot shows a REST client interface with a POST request to `http://127.0.0.1:8000/login_jwt`. The request body is a JSON object with `"username": "Sonam"` and `"password": "$2b$12$ocKMz3Aak7cTH2h/00zyNeEeS6weIyK7wGpM.xnqQgbaaac2kw5E0"`. The response body is a JSON object with `"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6I1NvbWVtIn0.XXHDELtUCQwBxz-RTUU7PsisvGIHWtVE5vDYmFQfT8J8"`. The status bar at the bottom indicates a 200 OK response.

```
POST http://127.0.0.1:8000/login_jwt

{
  "username": "Sonam",
  "password": "$2b$12$ocKMz3Aak7cTH2h/00zyNeEeS6weIyK7wGpM.xnqQgbaaac2kw5E0"
}
```

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6I1NvbWVtIn0.XXHDELtUCQwBxz-RTUU7PsisvGIHWtVE5vDYmFQfT8J8"
}
```

If user doesn't exists:

POST http://127.0.0.1:8000/login_jwt

Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "username" : "onam",
3   "password" : "onam@123"
4 }
```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "detail": "User not found"
3 }
```

Task 9: Create GET API with Pagination Requirements:

Fetch staff records using:

- skip
- limit Example: /staff?skip=0&limit=5

Only first 5 records will be shown:

GET ☐ http://127.0.0.1:8000/hospitalstaff_pagination?skip=0&limit=5

Params ☒ Authorization Headers 6 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ☐

1

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize JSON ☐

```
22     },
23     {
24       "role": "Doctor",
25       "id": 4,
26       "password": "rupesh@123",
27       "username": "Rupesh",
28       "department": "Padiatrics"
29     },
30     {
31       "role": "Front Desk",
32       "id": 5,
33       "password": "karan@123",
34       "username": "Karan",
35       "department": "Reception"
36     }
37   ]
```

Task 10: Create Search API Requirements:

Search staff records by:

- username Example: /search?username=amit

GET http://127.0.0.1:8000/hospitalstaff_s/search?username=Ram

Params Authorization Headers 6 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

1

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "role": "Coordinator",
4     "id": 6,
5     "password": "ram@123",
6     "username": "Ram",
7     "department": "Reception"
8   }
9 ]
```

Task 11: Create Filtering API Requirements: Filter staff records by:

- department Example: /filter?department=Cardiology

GET

▼

http://127.0.0.1:8000/hospitalstaff_f/filter?department=Cardiology

Params

Authorization

Headers

6

Body

Scripts

Settings

Query Params

Key	Value	Description
✓ department	Cardiology	
Key	Value	Description

Body	Cookies	Headers	4	Test Results	200 OK
Pretty	Raw	Preview	Visualize	JSON	
<pre>1 [2 { 3 "password": "vivek@123", 4 "id": 3, 5 "role": "Doctor", 6 "username": "Vivek", 7 "department": "Cardiology" 8 } 9]</pre>					

Task 12: Implement Role-Based Access Control (RBAC) Requirements:

Only Admin can:

- Add Staff
- Update Staff
- Delete Staff

Doctor & Receptionist can:

- View Staff Records

Delete Staff:

Access Denied if the user is not an Admin/Director:

DELETE	http://127.0.0.1:8000/delete_role_rb/4?username=Rohan
Params	Authorization Headers 6 Body Scripts Settings
Query Params	
Key	Value
✓ username	Rohan
Key	Value

Body	Cookies Headers 4 Test Results
Pretty	Raw Preview Visualize JSON
1	{
2	"detail": "Access Denied"
3	}

Access Granted if the user is not an Admin/Director:

DELETE	http://127.0.0.1:8000/delete_role_rb/4?username=Nitesh
Params	Authorization Headers 6 Body Scripts Settings
☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON	
1	
Body	Cookies Headers 4 Test Results
Pretty	Raw Preview Visualize JSON
1	{
2	"message": "Role Deleted Successfully"
3	}

• Add Staff:

Access Granted if the user is not an Admin/Director:

POST ▼ http://127.0.0.1:8000/assign_role_rb?username=Nitesh

Params ● Authorization Headers 8 **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "id" : 9,
3   "username" : "Yogita",
4   "password" : "yogita@123",
5   "role" : "Coordinator",
6   "department" : "Reception"
7 }
```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize **JSON** ▼

```
1 {
2   "message": "Role assigned Successfully"
3 }
```

Access Denied if the user is not an Admin/Director:

POST ▼ http://127.0.0.1:8000/assign_role_rb?username=Ram

Params ● Authorization Headers 8 **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "id" : 10,
3   "username" : "Yogita",
4   "password" : "yogita@123",
5   "role" : "Coordinator",
6   "department" : "Reception"
7 }
```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize **JSON** ▼

```
1 {
2   "detail": "Access Denied"
3 }
```

• Update Staff :

Access Denied:

PUT http://127.0.0.1:8000/update_role_rb/7?username=Vivek

Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "id" : 7,
3   "username" : "Rahul",
4   "password" : "rahul@123",
5   "role" : "Coordinator",
6   "department" : "Reception"
7 }
```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "detail": "Access Denied"
3 }
```

Access Granted:

PUT http://127.0.0.1:8000/update_role_rb/7?username=Nitesh

Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "id" : 7,
3   "username" : "Rahul",
4   "password" : "rahul@123",
5   "role" : "Coordinator",
6   "department" : "Reception"
7 }
```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Role Updated Successfully"
3 }
```

Detailed Database with all above changes:

Data Output Messages Notifications						
	id [PK] integer	username character varying	password character varying	role character varying	department character varying	
1	1	Nitesh	nitesh@123	Director	Clinical	
2	2	Shiksha	shiksha@123	Doctor	Support & Diagnos...	
3	3	Vivek	vivek@123	Doctor	Cardiology	
4	5	Karan	karan@123	Front Desk	Reception	
5	6	Ram	ram@123	Coordinator	Reception	
6	7	Rahul	rohan@123	Coordinator	Orthopedics	
7	8	Sonam	\$2b\$12\$ocKMr3Aak7cTH2h/00zyNeEeS6welyK7wGpM.xnqQgbaaac2kw...	Doctor	Neurology	
8	9	Yogita	yogita@123	Coordinator	Reception	

Task 13: Create Admin-Only API Requirements: Protect at least one API using role verification.

Example:

- Add Staff OR
- Delete Staff

Delete Staff:

Access Denied if the user is not an Admin/Director:

DELETE	http://127.0.0.1:8000/delete_role_rb/4?username=Rohan
Params	Authorization Headers 6 Body Scripts Settings
Query Params	
Key	Value
☒ username	Rohan
Key	Value

Body	Cookies	Headers 4	Test Results
Pretty	Raw	Preview	Visualize
JSON			
1	{		
2	"detail": "Access Denied"		
3	}		

Access Granted if the user is not an Admin/Director:

DELETE ▼ http://127.0.0.1:8000/delete_role_rb/4?username=Nitesh

Params ● Authorization Headers 6 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▼

```
1
```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize JSON ▼ ≡

```
1 {
2   "message": "Role Deleted Successfully"
3 }
```

- Add Staff:

Access Granted if the user is not an Admin/Director:

POST ▼ http://127.0.0.1:8000/assign_role_rb?username=Nitesh

Params ● Authorization Headers 8 **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▼

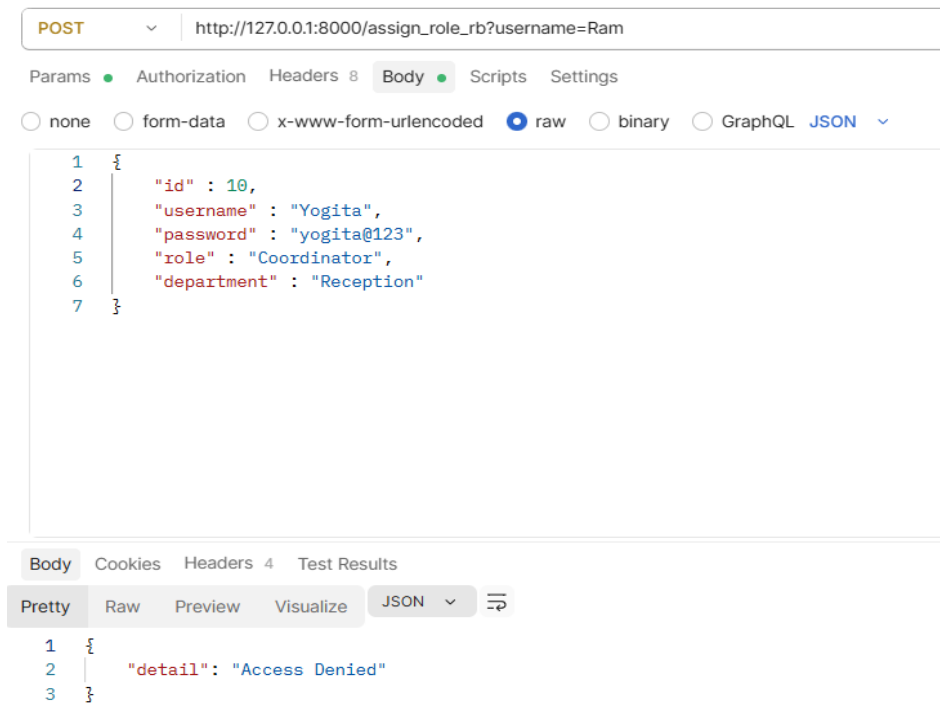
```
1 {
2   "id" : 9,
3   "username" : "Yogita",
4   "password" : "yogita@123",
5   "role" : "Coordinator",
6   "department" : "Reception"
7 }
```

Body Cookies Headers 4 Test Results

Pretty Raw Preview Visualize JSON ▼ ≡

```
1 {
2   "message": "Role assigned Successfully"
3 }
```

Access Denied if the user is not an Admin/Director:



Task 14: Testing Demonstrate:

- User Registration
- Login
- JWT Token Generation
- Password Hashing
- Search API
- Filtering API
- Pagination API
- Role-Based Access Control

Everything is tested and screenshots added above.